

tf.identity_n Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/identity_n

Returns a list of tensors with the same shapes and contents as the input

Function Signatures

```
tf.identity_n( input, name=None )  
with tf.get_default_graph().gradient_override_map(  
    {'IdentityN': 'OverrideGradientWithG'}): y, _ =  
    identity_n([f(x), x])  
@tf.RegisterGradient('OverrideGradientWithG') def ApplyG(op,  
    dy, _): return [None, g(dy)] # Do not backprop to f(x).
```

Code Examples

```
tf.identity_n(  
    input, name=None  
)
```

```
tf.identity_n(  
    input, name=None  
)
```

```
with tf.get_default_graph().gradient_override_map(  
    {'IdentityN': 'OverrideGradientWithG'}):  
    y, _ = identity_n([f(x), x])  
    @tf.RegisterGradient('OverrideGradientWithG')  
    def ApplyG(op, dy, _):  
        return [None, g(dy)] # Do not backprop to f(x).
```

```
with tf.get_default_graph().gradient_override_map(  
    {'IdentityN': 'OverrideGradientWithG'}):  
    y, _ = identity_n([f(x), x])  
    @tf.RegisterGradient('OverrideGradientWithG')  
    def ApplyG(op, dy, _):  
        return [None, g(dy)] # Do not backprop to f(x).
```

Documentation

Returns a list of tensors with the same shapes and contents as the input

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.identity_n`

tensors.

This op can be used to override the gradient for complicated functions. For example, suppose $y = f(x)$ and we wish to apply a custom function g for backprop such that $dx = g(dy)$. In Python,

Returns